

 Защита конечных точек для разработчиков уже здесь: находите все учетные данные на каждом компьютере разработчика раньше злоумышленников. [👉 УЗНАТЬ БОЛЬШЕ](#)

ЛУЧШИЕ ПРАКТИКИ

 GitGuardian **BLOG**

ВЕБ-САЙТ

ИНТЕРАКТИВНАЯ ДЕМОНСТРАЦИЯ

УЧЕБНЫЙ ЦЕНТР

NH1

защиты секретов в эпоху ИИ-помощников в программировании

Современные среды разработки раскрывают конфиденциальную информацию в файлах, подсказках, журналах и командах. Узнайте, как многоуровневые локальные средства контроля снижают риски, связанные с хранением секретов.



ДУЭЙН МАКДЭНИЕЛ

4 МАЯ 2026 ГОДА · 8 МИНУТ ЧТЕНИЯ

Следуйте за нами дальше



🖥️ Защита конечных точек для разработчиков уже здесь: находите все учетные данные на каждом компьютере разработчика раньше злоумышленников. 🙌 [УЗНАТЬ БОЛЬШЕ](#)

Local Guardrails

GitGuardian **BLOG**

ВЕБ-САЙТ

ИНТЕРАКТИВНАЯ ДЕМОНСТРАЦИЯ

УЧЕБНЫЙ ЦЕНТР

NHI

Security

In the Age of AI Coding Assistants

Содержание

Общая проблема — кража учетных данных

На рабочей станции теперь хранится слишком много данных, чтобы их можно было игнорировать

Ранние контрольные точки снижают ущерб

Защита секретов ваших разработчиков

Инструменты искусственного интеллекта нуждаются в ограничениях на этапах передачи данных

Многоуровневая защита без создания отдельного рабочего процесса

Раньше казалось, что безопасность цепочки поставок программного обеспечения — это проблема, которая существует где-то далеко.

В первую очередь в поле зрения попали репозитории и системы сборки. Также в центре внимания оказались реестры пакетов, конвейеры непрерывной интеграции и непрерывной поставки, автоматизация выпуска, облачные платформы и хранилища артефактов. Все это по-прежнему важно и нуждается в защите, но теперь уязвимые места находятся ближе к тому, с чем разработчики работают каждый день.

 Защита конечных точек для разработчиков уже здесь: находите все учетные данные на каждом компьютере разработчика раньше злоумышленников. 📌 [УЗНАТЬ БОЛЬШЕ](#)

Последствия для безопасности легко недооценить. Современная рабочая станция взаимодействует с исходным кодом, менеджерами пакетов,

 [GitGuardian](#) **BLOG** [ВЕБ-САЙТ](#) [ИНТЕРАКТИВНАЯ ДЕМОНСТРАЦИЯ](#) [УЧЕБНЫЙ ЦЕНТР](#) [NHI](#)

помощниками по написанию кода на основе искусственного интеллекта, терминалами, локальными кэшами и файлами окружения. Именно на ней создаются, копируются, тестируются, регистрируются и слишком часто забываются учетные данные.

Злоумышленники это понимают. Они ищут не только уязвимые производственные сервисы или скомпрометированные этапы сборки. Они ищут материалы для доступа, которые позволяют одной системе доверять другой.

Нам нужно обновить наши модели защиты. Безопасность не может ждать, пока код попадет в удаленный репозиторий или конвейер. К тому времени учетные данные могут уже оказаться в истории Git, в подсказке для модели, в локальном журнале, в артефакте сборки или в скрипте установки пакета.

Точка контроля должна смещаться на более ранние этапы процесса создания программного обеспечения.

Общая тенденция — кража учетных данных

[Недавнее исследование компании GitGuardian](#) указывает на четкую закономерность в атаках на цепочки поставок программного обеспечения: злоумышленники все чаще нацеливаются на учетные данные, используемые в рабочих процессах разработчиков.

В апреле 2026 года мы проанализировали три кампании по взлому цепочек поставок, которые затронули [npm](#), [PyPI](#) и [Docker Hub в течение 48 часов](#).

Экосистемы и методы были разными, но цель оставалась неизменной.

Каждая кампания была направлена на кражу полезных учетных данных из сред разработчиков или конвейеров непрерывной интеграции и доставки.

 Защита конечных точек для разработчиков уже здесь: находите все учетные данные на каждом компьютере разработчика раньше злоумышленников. 📌 [УЗНАТЬ БОЛЬШЕ](#)

публикации зараженных версии пакетов, к которым могла получить доступ жертва. В ходе кампании на PyPI злоумышленники похищали защищенные ключи командной строки, учетные данные для доступа к

 [GitGuardian](#) **BLOG** [ВЕБ-САЙТ](#) [ИНТЕРАКТИВНАЯ ДЕМОНСТРАЦИЯ](#) [УЧЕБНЫЙ ЦЕНТР](#) [NHI](#)

использование для проникновения в следующую систему.

Именно это делает проблему настолько серьезной.

Разработчики — привлекательная цель для злоумышленников

У разработчика может быть доступ к системе контроля версий, облачным аккаунтам, реестрам пакетов, хранилищам артефактов, промежуточным средам, инструментам для реагирования на инциденты и внутренним интерфейсам прикладного программирования. У сборщика может быть доступ к учетным данным для развертывания, токенам для публикации пакетов и инфраструктуре, связанной с производственным процессом. Один открытый токен может стать связующим звеном между несколькими уровнями процесса доставки.

That is why credential exposure is different from many other bugs. An attacker does not always need to exploit a software flaw, maintain persistence, or modify production code. Sometimes, authenticated access is enough. Sometimes, a short-lived foothold on a developer machine can uncover a credential with broader reach.

[The GitGuardian 2026 State of Secrets Sprawl report](#) makes the scale of the situation clear. We found over 28.6 million new secrets detected in public GitHub commits in 2025, a 34 percent year-over-year increase. Internal repositories are roughly six times more likely than public repositories to contain hardcoded credentials. About 28% of incidents originate entirely outside repositories, in collaboration systems such as Slack, Jira, and Confluence.

The repository is no longer the boundary. If a secret can be found on the machine in plaintext, it is likely to spread elsewhere in plaintext and be usable by anyone who finds it.

 Защита конечных точек для разработчиков уже здесь: находите все учетные данные на каждом компьютере разработчика раньше злоумышленников. 🙌 [УЗНАТЬ БОЛЬШЕ](#)

They hold source trees, dotfiles, shell history, local environment files, integrated

 [GitGuardian](#) **BLOG** [ВЕБ-САЙТ](#) [ИНТЕРАКТИВНАЯ ДЕМОНСТРАЦИЯ](#) [УЧЕБНЫЙ ЦЕНТР](#) [NHI](#)

involved during normal review. Many never leave the machine, some slip in artifacts that developers rarely inspect.

That makes local exposure difficult to manage with repository-only controls.

A credential can appear in a `.env` file, get printed into terminal history, land inside a test config, show up in build output, or be copied into an AI prompt during troubleshooting. None of that requires a malicious commit. None of it necessarily triggers a centralized scanner. Yet each moment can create real access risk.

We need, as an industry, to scan the places where credentials are collected outside Git. Project workspaces, dotfiles, build output, and agent folders can all store copied tokens, configuration blocks, troubleshooting output, and cached context. Attackers harvest this local data because it can lead directly to valid access.

The Shai-Hulud data gives that concern weight. Across [6,943 compromised machines, it found 33,185 unique credentials](#), with at least 3,760 still valid when first checked. That is not a theoretical workstation problem. It is a practical attacker workflow.

Compromise the machine. Search the context. Extract the access. Move on.

The workstation has become a security boundary because so many tools assume the local environment is trusted. Package managers run install scripts. Extensions read project files. Terminals expose environment variables. Local automation touches real systems. AI agents can read files, run commands, and summarize outputs.

Each of those actions may be useful, but each can also become a path for accidental exposure or malicious instruction.

AI-assisted development adds a newer layer to the same problem. AI coding tools now work closer to the developer's files, terminal, editor, and environment variables. A prompt can contain a credential. A tool can call and read a sensitive file. A generated

 Защита конечных точек для разработчиков уже здесь: находите все учетные данные на каждом компьютере разработчика раньше злоумышленников. 🙌 [УЗНАТЬ БОЛЬШЕ](#)

The exposure surface is no longer just human typing plus code review. It now includes the interaction between humans, local tools, automated agents, and external services. And

 [GitGuardian](#) **BLOG**

[ВЕБ-САЙТ](#)

[ИНТЕРАКТИВНАЯ ДЕМОНСТРАЦИЯ](#)

[УЧЕБНЫЙ ЦЕНТР](#)

[NHI](#)

[commit](#).

Security controls have to meet the workflow where it actually happens.

Earlier Checkpoints Reduce Damage

Traditional supply chain controls still matter. Teams still need source control protections, dependency scanning, secure continuous integration and delivery, artifact integrity, release controls, and production deployment guardrails.

But those controls often fire after the risky moment has already happened.

A developer may have created a local file with a credential. An AI assistant may have received sensitive context. A package install script may have read environment variables. A token may have entered local Git history before it ever reaches a remote repository.

Rotating a credential after it reaches a shared repository can become a full incident response exercise. Someone has to identify ownership, revoke the credential, issue a replacement, check usage, test dependent applications, review access, clean history where possible, and document the event. That work is necessary, but it is expensive.

Catching the same issue while the developer is still editing a file is simpler. Remove it. Replace it with a safe reference. Keep moving.

The strongest model treats credential detection as a continuous developer-side control, not an occasional cleanup task. The tool has to sit where developers already work: in the editor, in Git hooks, in the terminal, and inside AI coding workflows.

Protecting Your Developers' Secrets

[ggshield](#) is the [GitGuardian](#) command-line interface for scanning developer workflows.

You can run [ggshield](#) locally or in continuous integration environments, where it provides

 Защита конечных точек для разработчиков уже здесь: находите все учетные данные на каждом компьютере разработчика раньше злоумышленников. 🙌 [УЗНАТЬ БОЛЬШЕ](#)

A local scan catches problems before code moves into shared infrastructure. A continuous integration scan catches problems after code leaves the laptop. A pre-

 [GitGuardian](#) **BLOG** [ВЕБ-САЙТ](#) [ИНТЕРАКТИВНАЯ ДЕМОНСТРАЦИЯ](#) [УЧЕБНЫЙ ЦЕНТР](#) [NHI](#)

into a separate security process.

Git Hooks Add Another Layer Of Protection

[Using gghshield pre-commit hooks](#) means a scan runs before Git creates a commit. Teams can configure it through the pre-commit framework, install it locally for specific repositories, or install it globally across current and future repositories on a developer workstation.

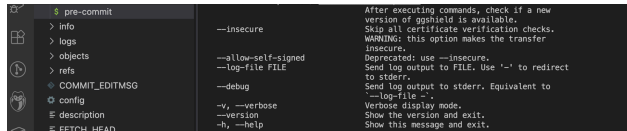
The global option is important. Not every leak happens in the main codebase. Temporary repositories, test folders, side projects, cloned examples, and one-off experiments all create exposure. A repository-by-repository rollout leaves gaps. A global hook gives the developer machine a broader default.

[A pre-push hook](#) catches a later moment. It runs before code leaves the machine for a remote repository. GitGuardian documents local, framework-based, and global installation modes for this control as well. Together, pre-commit and pre-push hooks create two useful gates: one before local history becomes durable, and one before code reaches shared infrastructure.

Finding Secrets On Save

[GitGuardian's VS Code extension](#) uses the bundled ggshield command-line interface to scan code as developers write or modify it. A scan is run automatically on saving a file. Findings are shown instantly and directly inside the editor through code annotations, status bar warnings, and the Problems panel. This extension also works with Cursor, Antigravity, and Windsurf.

 Защита конечных точек для разработчиков уже здесь: находите все учетные данные на каждом компьютере разработчика раньше злоумышленников. [👉 УЗНАТЬ БОЛЬШЕ](#)

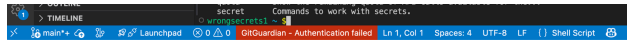


ВЕБ-САЙТ

ИНТЕРАКТИВНАЯ ДЕМОНСТРАЦИЯ

УЧЕБНЫЙ ЦЕНТР

NHI



GitGuardian VS Code Extension in action

Security controls fail when they are too late, too noisy, or too far away from the mistake. A good local control gives feedback in context. It explains what happened. It helps the developer fix the issue before it becomes a ticket, a broken build, or an incident.

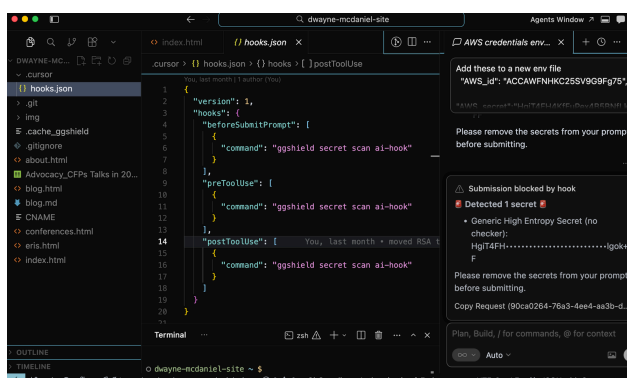
Ai Tools Need Guardrails At The Handoff Points

AI coding tools deserve special attention because they change where leakage can occur.

An AI workflow may expose sensitive material before code exists as a file. A developer might paste a credential into a prompt while debugging. An agent might read a local configuration file. A tool call might execute a command that prints environment variables. Output from that command might then move into the model context or local logs.

That is a different path than traditional source code leakage.

GitGuardian's AI coding tools integration addresses this by placing controls inside the hook systems of tools such as Cursor, Claude Code, and VS Code with GitHub Copilot. The integration scans three stages: prompt submission, pre-tool use, and post-tool use.





Защита конечных точек для разработчиков уже здесь: находите все учетные данные на каждом компьютере разработчика раньше злоумышленников. 🙌 [УЗНАТЬ БОЛЬШЕ](#)

and model context protocols can detect execution, blocking risky actions before they run. Post-tool use scanning checks outputs after execution and sends a desktop notification when credentials appear.



BLOG

ВЕБ-САЙТ

ИНТЕРАКТИВНАЯ ДЕМОНСТРАЦИЯ

УЧЕБНЫЙ ЦЕНТР

NHI

The risky moment may be a prompt. It may be a file read. It may be a shell command. It may be the output of a tool the developer did not manually inspect. A repository-only control sees too little of this flow. A hook inside the AI workflow can stop exposure at the handoff point.

The editor catches the issue while the developer writes. AI hooks catch sensitive material before prompts, tool calls, or outputs move it somewhere risky. Git hooks catch credentials before they enter commit history or leave the laptop. Continuous integration and server-side controls provide backup once code reaches shared systems.

Layered Prevention Without Forcing A Separate Workflow

Developer environments are now connected, automated, and increasingly assisted by tools that can act on local context. Security has to account for that reality. Waiting for a remote scan is too late for credential exposure.

The better model is straightforward: find credentials earlier, block them closer to where they appear, and reduce the chance that a developer's laptop becomes the easiest path into the software supply chain.

GitGuardian's ggshield, IDE extensions, AI hooks, and Git hooks all point toward that model. They bring detection into the places developers already use, rather than asking developers to leave their workflow for security. They reduce the time between mistakes and feedback. They give teams a consistent detection engine across local development, AI-assisted coding, Git workflows, and automation.

The supply chain now includes the workstation.

Treat it that way.

🖥️ Защита конечных точек для разработчиков уже здесь: находите все учетные данные на каждом компьютере разработчика раньше злоумышленников. 🙌 [УЗНАТЬ БОЛЬШЕ](#)



ВЕБ-САЙТ

ИНТЕРАКТИВНАЯ ДЕМОНСТРАЦИЯ

УЧЕБНЫЙ ЦЕНТР

NHI

Share this article on [Twitter](#), [HackerNews](#), [LinkedIn](#), or [Reddit](#).

Related Articles



BREACH EXPLAINED

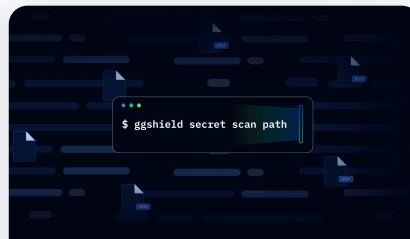
BEST PRACTICES

How GitGuardian Enables Rapid Response to the LiteLLM Supply Chain Attack

Learn how to detect compromise, assess your exposure to the LiteLL...



GUILLAUME VALADON
25 MAR 2026 – 9 MIN READ



BEST PRACTICES

Protecting Developers Means Protecting Their Secrets

Secrets don't just leak from Git. They accumulate in...



DWAYNE MCDANIEL
3 MAR 2026 – 9 MIN READ



CISO ROADMAP

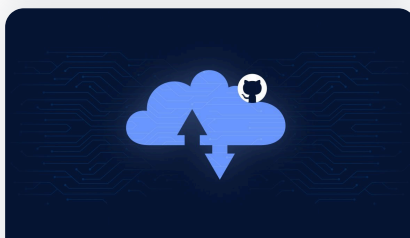
BEST PRACTICES

Confronting Vault Sprawl And The Risks It Brings

Vault sprawl means duplicated secrets, fragmented access, and...



DWAYNE MCDANIEL
23 FEB 2026 – 8 MIN READ



 Защита конечных точек для разработчиков уже здесь: находите все учетные данные на каждом компьютере разработчика раньше злоумышленников. 🙌 [УЗНАТЬ БОЛЬШЕ](#)

The Ultimate Guide to GitHub Backups



GitGuardian **BLOG**

ВЕБ-САЙТ

ИНТЕРАКТИВНАЯ ДЕМОНСТРАЦИЯ

УЧЕБНЫЙ ЦЕНТР

NHI

and more important to...



GUEST EXPERT

25 NOV 2025 – 12 MIN
READ

Start your journey to secrets-free source code

AND KEEP YOUR SECRETS OUT OF SIGHT

Start for free

Book a demo

🖥️ Защита конечных точек для разработчиков уже здесь: находите все учетные данные на каждом компьютере разработчика раньше злоумышленников. 🙌 [УЗНАТЬ БОЛЬШЕ](#)



PLATFORM

DEVELOPERS

RESOURCES

COMPANIES



GitGuardian

BLOG

ВЕБ-САЙТ

ИНТЕРАКТИВНАЯ ДЕМОНСТРАЦИЯ

УЧЕБНЫЙ ЦЕНТР

ННН

With automated secrets detection and remediation, our platform enables Dev, Sec, and Ops to advance together towards the Secure Software Development Lifecycle.

Subscribe to our newsletter to receive the latest content and updates from GitGuardian.

Email address

By submitting this form, I agree to GitGuardian's [Privacy Policy](#)

Public Monitoring
GITGUARDIAN VS

GitHub Advanced Security

GitLab Secret Detection

truffleHog v3

More alternatives

API Documentation

Good Samaritan

Labs

GitHub

Roadmap

API Status

Security Champions

Dev & Sec resources

State of Secrets
Sprawl

Events

GitHub Security Audit

PRICING

Plans

Value Calculator

Careers

Contact

FAQ

Partners

Newsroom

© 2026 GitGuardian. All Rights Reserved.

[Term & Conditions](#)

[Privacy Policy](#)

[Public Security](#)

[Policy](#) [Cookies](#)